Alternative Assessment Tool (AAT) Report on

"Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

[Course Code: 23ECSPE1AD]

Submitted by

Ananya Lakshmi Narasimhan **1BF24EC016**

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

2026 _ 2027

1. Objective

To execute one combinational module and one sequential module using Verilog HDL on the Cadence tool and demonstrate their correct operation through simulation waveforms.

The combinational module selected is a 4-bit magnitude comparator. The sequential module selected is a 4-bit ring counter.

2. Software and Tools Used

Cadence digital simulation environment (SimVision waveform viewer), Verilog HDL, and a testbench for each design.

3. Combinational Module – 4-bit Magnitude Comparator

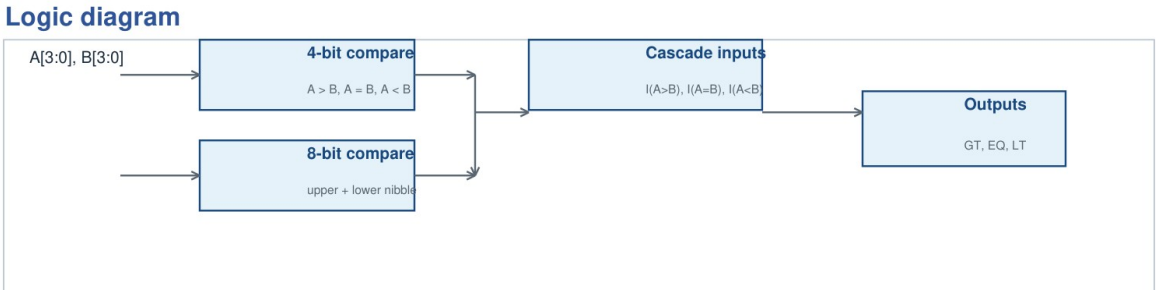
3.1 Aim and Description

To design and simulate a 4-bit magnitude comparator that compares two 4-bit unsigned binary inputs A and B. It produces three mutually exclusive outputs: GT (A is greater than B), EQ (A is equal to B), and LT (A is less than B). The design also includes cascade inputs so that comparator stages can be connected to form a wider comparator. For this stand-alone demonstration, gt_in=0, eq_in=1 and lt_in=0.

3.2 Truth Table

Condition	GT	EQ	LT
A > B	1	0	0
A = B	0	1	0
A < B	0	0	1

3.3 Design Diagram



The design diagram shows the 4-bit comparison logic, cascade inputs and the GT, EQ and LT outputs. The Verilog module implements the 4-bit stage; the diagram also illustrates how cascade outputs can be used to build a wider comparator.

3.4 Working Principle / Mechanism

Each corresponding bit of A and B is compared. The expression x = ~(a ^ b) acts as a bit-wise XNOR: x[i] becomes 1 when a[i] and b[i] are equal. If all four XNOR outputs are 1, eq_w becomes 1, indicating equality.

The greater-than logic checks the bits from the most significant bit (MSB) towards the least significant bit (LSB). A is greater if the first differing bit has A=1 and B=0. The less-than logic works in the opposite way. The cascade inputs affect the final outputs only when the local 4-bit values are equal.

For stand-alone operation, the cascade inputs are tied to gt_in=0, eq_in=1 and lt_in=0. Thus, the outputs directly indicate whether A is greater than, equal to, or less than B. Since it is combinational logic, the outputs change whenever A or B changes, after the propagation delay.

3.5 Verilog Design Code

```
`timescale 1ns/1ps
// 4-bit cascadable magnitude comparator
// Stand-alone use: gt_in = 0, eq_in = 1, lt_in = 0

module comparator_4bit(
    input [3:0] a, b,
    input gt_in, eq_in, lt_in,
    output gt, eq, lt
);

wire [3:0] x = ~(a ^ b);
wire eq_w = &x;

wire gt_w = (a[3] & ~b[3])
          | (x[3] & a[2] & ~b[2])
          | (x[3] & x[2] & a[1] & ~b[1])
          | (x[3] & x[2] & x[1] & a[0] & ~b[0]);

wire lt_w = (~a[3] & b[3])
          | (x[3] & ~a[2] & b[2])
          | (x[3] & x[2] & ~a[1] & b[1])
          | (x[3] & x[2] & x[1] & ~a[0] & b[0]);

assign gt = gt_w | (eq_w & gt_in);
assign eq = eq_w & eq_in;
assign lt = lt_w | (eq_w & lt_in);

endmodule
```

3.6 Testbench

```
`timescale 1ns/1ps

module comparator_4bit_tb;

reg [3:0] a, b;
wire gt, eq, lt;

// Cascade inputs tied off for stand-alone use
comparator_4bit uut (
    .a(a), .b(b),
    .gt_in(1'b0), .eq_in(1'b1), .lt_in(1'b0),
    .gt(gt), .eq(eq), .lt(lt)
);

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, comparator_4bit_tb);
    $monitor("t=%0d ns a=%0d b=%0d -> gt=%b eq=%b lt=%b",
              $time, a, b, gt, eq, lt);

    a = 4'd5;  b = 4'd3;  #10;
    a = 4'd3;  b = 4'd5;  #10;
    a = 4'd7;  b = 4'd7;  #10;
    a = 4'd8;  b = 4'd7;  #10;
    a = 4'd0;  b = 4'd15; #10;
    a = 4'd15; b = 4'd15; #10;

    $finish;
end

endmodule
```

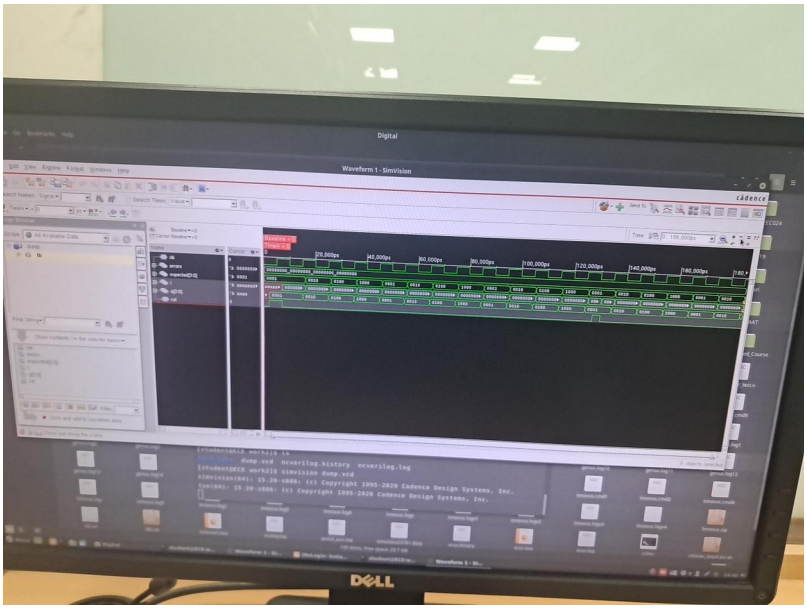
3.7 Testbench Operation

The testbench instantiates comparator_4bit and fixes the cascade inputs for stand-alone use. It applies six input pairs at 10 ns intervals, generates dump.vcd for waveform viewing, and uses $monitor to print the input values and the three comparison outputs whenever a monitored signal changes.

3.8 Applied Test Cases and Expected Results

A (decimal)	B (decimal)	Relation	GT	EQ	LT	Meaning
5	3	A>B	1	0	0	GT asserted
3	5	A<B	0	0	1	LT asserted
7	7	A=B	0	1	0	EQ asserted
8	7	A>B	1	0	0	GT asserted
0	15	A<B	0	0	1	LT asserted
15	15	A=B	0	1	0	EQ asserted

3.9 Cadence Waveform and Observation



The comparator waveform shows the changing input buses A and B and the corresponding output signals GT, EQ and LT. At each stable input interval, exactly one comparison output should be HIGH: GT for A>B, EQ for A=B, or LT for A<B. As the testbench changes the input pair, the asserted output changes accordingly. The six directed test cases listed above are the reference for interpreting the simulation.

Because the screenshot is a photograph of the SimVision display, some bus labels are small. The expected output sequence for the provided testbench is 100, 001, 010, 100, 001, 010.

4. Sequential Module – 4-bit Ring Counter

4.1 Aim and Description

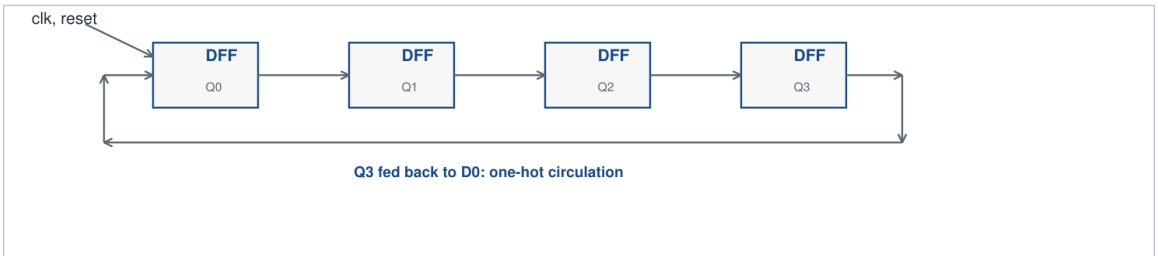
To design and simulate a 4-bit one-hot ring counter. It consists of four flip-flop stages connected as a circular shift register. A single logic 1 circulates through the four output bits on successive rising edges of the clock.

4.2 State Sequence / Function Table

Clock count	Q[3:0]	State
Reset / 0	0001	Initial one-hot state
1	0010	1 shifts left
2	0100	1 shifts left
3	1000	1 reaches MSB
4	0001	MSB fed back to LSB

4.3 Design Diagram

Logic diagram



The ring-counter logic diagram shows four D flip-flops connected in a shift chain, with the Q3 output fed back to the D input of Q0. All flip-flops share the same clock.

4.4 Working Principle / Mechanism

The counter uses four flip-flops and stores a one-hot pattern, meaning exactly one output bit is HIGH at a time. The asynchronous active-high reset sets q to 0001 immediately when rst becomes HIGH, without waiting for a clock edge.

On every positive edge of clk, the bits shift in the order q <= {q[N-2:0], q[N-1]}. Therefore, the sequence is 0001 → 0010 → 0100 → 1000 → 0001 and repeats. The most significant bit is fed back into the least significant position, completing the ring. Four clock edges are required for one complete rotation.

4.5 Verilog Design Code

```
`timescale 1ns/1ps

// 4-bit Ring Counter
// Sequence: 0001 -> 0010 -> 0100 -> 1000 -> 0001
// Asynchronous active-high reset

module ring_counter #(parameter N = 4) (
    input clk,
    input rst,
    output reg [N-1:0] q
);

always @(posedge clk or posedge rst) begin
    if (rst)
        q <= {{(N-1){1'b0}}, 1'b1};
    else
        q <= {q[N-2:0], q[N-1]};
end
```

```
endmodule
```

4.6 Testbench

```
`timescale 1ns/1ps

// Self-checking testbench for 4-bit ring counter
// Expected final PASS message:
// RING COUNTER TB : PASSED

module tb;

reg clk;
reg rst;
wire [3:0] q;
reg [3:0] expected;
integer i;
integer errors;

// Instantiate the design under test
ring_counter #(.N(4)) uut (
    .clk(clk),
    .rst(rst),
    .q(q)
);

// Clock period = 10 ns
initial clk = 1'b0;
always #5 clk = ~clk;

// Check the counter after each rising edge
task check_next_state;
begin
    @(posedge clk);
    #1;
    expected = {expected[2:0], expected[3]};

    if (q !== expected) begin
        errors = errors + 1;
        $display(
            "FAIL at %0t: q=%b, expected=%b",
            $time, q, expected
        );
    end

    // Verify that exactly one bit is HIGH
    if ((q == 4'b0000) ||
        ((q & (q - 4'b0001)) != 4'b0000)) begin
        errors = errors + 1;
        $display(
            "FAIL at %0t: output is not one-hot, q=%b",
            $time, q
        );
    end

    $display("%4t ns : q = %b", $time, q);
end
endtask

initial begin
    errors = 0;
    expected = 4'b0001;
    rst = 1'b0;

    $dumpfile("dump.vcd");
```

```
    $dumpvars(0, tb);

    // Apply asynchronous reset
    #2 rst = 1'b1;
    #2;
    if (q !== 4'b0001) begin
        errors = errors + 1;
        $display("FAIL: reset value q=%b", q);
    end
    else begin
        $display("%4t ns : q = %b (RESET)", $time, q);
    end

    // Release reset between clock edges
    #3 rst = 1'b0;

    // Test three complete rotations
    for (i = 0; i < 12; i = i + 1)
        check_next_state;

    // Assert asynchronous reset between clock edges
    #2 rst = 1'b1;
    #1;
    if (q !== 4'b0001) begin
        errors = errors + 1;
        $display("FAIL: asynchronous reset q=%b", q);
    end
    else begin
        $display("%4t ns : q = %b (ASYNC RESET)", $time, q);
    end

    // Release reset and restart the expected sequence
    #2 rst = 1'b0;
    expected = 4'b0001;

    // Verify six additional clock transitions
    for (i = 0; i < 6; i = i + 1)
        check_next_state;

    // Final test result
    if (errors == 0)
        $display("RING COUNTER TB : PASSED");
    else
        $display("RING COUNTER TB : FAILED (%0d errors)", errors);

    $finish;
end

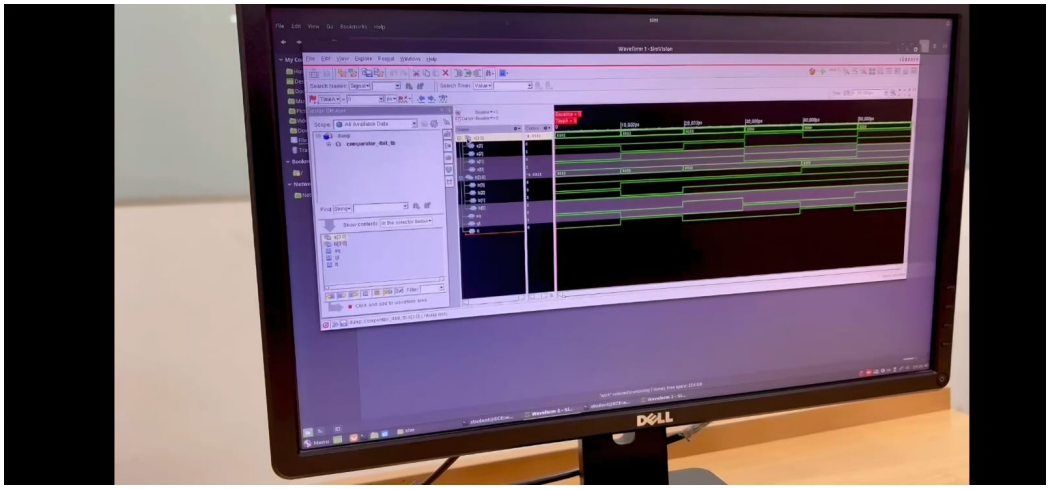
endmodule
```

4.7 Testbench Operation

The testbench creates a 10 ns clock (5 ns HIGH and 5 ns LOW), applies an asynchronous reset, and checks that the output is 0001. It then checks 12 rising-edge transitions, corresponding to three complete rotations. At each step, it compares q with the expected next state and verifies the one-hot property.

The testbench next asserts reset between clock edges to verify asynchronous reset operation. After releasing reset, it checks six more clock transitions. It prints a PASS message only if no errors are detected.

4.8 Cadence Waveform and Observation



The ring-counter waveform displays the clock, reset and counter output. During reset, q is forced to 0001. After reset is released, the output changes only at positive clock edges and follows the repeating one-hot sequence 0001 → 0010 → 0100 → 1000 → 0001.

The waveform therefore demonstrates sequential operation: the present state is retained between active clock edges, and the next state is produced at each rising edge. The reset pulse applied between clock edges immediately returns the output to 0001, confirming the asynchronous active-high reset.

5. Result and Conclusion

The 4-bit magnitude comparator was selected as the combinational module and the 4-bit ring counter was selected as the sequential module. Their Verilog designs were simulated in the Cadence environment and their operation was examined using SimVision waveforms. The comparator produces the correct GT, EQ or LT indication for each applied input pair. The ring counter circulates a single HIGH bit through its four states and responds immediately to asynchronous reset. Thus, the working of one combinational and one sequential digital module was demonstrated.

Reference

1. Project repository: ADLD-AAT-ananya-lakshmi- https://github.com/ananyal130806-debug/ADLD-AAT-ananya-lakshmi-
2. Comparator design reference: comparator_4bit Verilog logic, using bitwise XNOR for equality and MSB-to-LSB priority logic for greater-than and less-than outputs.
3. Ring-counter design reference: ring_counter Verilog logic, using a 4-bit circular shift sequence with asynchronous active-high reset.